

Model (Year) Source	Architecture	Training Process	Main Features
T-KAN May 2024 [33]	Spline-based, learnable edge activations	Sliding window, gradient descent, pruning	High interpretability, concept drift detection
SKAN May 2024 [6]	Structured, smooth nested functions	RMSE minimization, data-efficient, extrapolation in sparse data regions	High interpretability, scalable, effective in sparse data, smooth function representation
Wav-KAN May 2024 [8]	Wavelet-based, learnable edge functions	Batch norm, AdamW optimizer, grid search for wavelets	High interpretability, multi-resolution analysis, noise robustness, efficient for high-dimensional data
TKANs May 2024 [9]	RKAN layers with LSTM gating, B-spline activations	Adam optimizer, RMSE loss, early stopping	High accuracy in long-term forecasting, stable training, effective memory management for sequential data
KAN2CD May 2024 [10]	Two-level KANs with learnable embeddings	Adam optimizer, B-spline KANs for efficiency	High interpretability, efficient training, competitive accuracy, suitable for cognitive diagnosis
FastKAN May 2024 [11]	Gaussian RBFs with layer normalization	Benchmarked on MNIST, 20 epochs	3.3x faster than KAN, simplified implementation, retains accuracy for high-dimensional functions
C-KAN June 2024 [12]	Spline-based, learnable convolutions	Gradient descent with regularization, grid updates	High efficiency, adaptable activations, competitive accuracy
MT-KAN June 2024 [33]	Spline-based, learnable edge activations with cross-variable interactions	Gradient descent, pruning for efficiency	High interpretability, improved multivariate forecasting
SigKAN June 2024 [15]	Gated Residual KAN, path signature layer	Adam optimizer, early stopping	Accurate short-term forecasting, stable, captures temporal dependencies in complex time series
GraphKAN June 2024 [34]	Spline-based, learnable edge activations	Cosine Annealing, LayerNorm, 200 epochs	High accuracy, effective for few-shot classification
GKAN June 2024 [35]	Learnable spline functions on edges	Semi-supervised with backpropagation	Efficient, accurate, adjustable parameters for large-scale graph data
KINN June 2024 [16]	Spline-based, B-spline activations	Gradient descent, meshless sampling, triangular integration	High interpretability, efficient for PDEs, handles multi-frequency components, low spectral bias
Rational KAN June 2024 [36]	Rational basis functions (Padé, Jacobi)	Gradient descent (L-BFGS, Adam)	High accuracy, effective for physics-informed tasks and complex approximations
PDE-KAN June 2024 [16]	Spline-based, B-spline activations with tanh normalization	Meshless sampling, triangular integration	Low spectral bias, efficient for multi-frequency, adaptable to complex PDEs

Source: DOI: 10.48550/arXiv.2411.06078

Model (Year) Source	Architecture	Training Process	Main Features
KAGNNs June 2024 [30]	KAN-based, spline-driven updates	Gradient descent with early stopping	High interpretability, strong expressiveness, suited for graph regression and classification
PIKANs June 2024 [37]	Single-layer KAN, polynomial activations (e.g., Chebyshev)	Adam optimizer, RBA for loss adjustment	High accuracy with fewer parameters, adjustable polynomial order, enhanced stability with double precision
DeepOKANs June 2024 [37]	Branch [128, 100, 100, 100], Trunk [4, 100, 100, 100], Chebyshev polynomials	Adam optimizer, 200k iterations, L2 regularization	Robust to noise, strong in complex tasks, higher computational cost
RLDK June 2024 [38]	Spline-based, compact architecture with learnable edge activations	LBFGS optimizer with custom loss (reconstruction and prediction)	High parameter efficiency, fast training, suitable for real-time control, data-efficient
TKAT June 2024 [39]	Encoder-decoder with TKAN layers, self-attention	Adam optimizer, MSE loss, early stopping	High interpretability, captures temporal dependencies, suited for multivariate time series
KAN-EEG June 2024 [40]	Spline-based, learnable edge activations	100 epochs on EEG data, with gradient descent and epoch-based convergence	High interpretability, efficient, adaptable across datasets, suitable for on-device deployment
KANQAS June 2024 [41]	Spline-based, learnable activations in Double Deep Q-Network (DDQN)	Gradient descent with RL in DDQN	High interpretability, parameter-efficient, effective for quantum state prep and quantum chemistry
KCN June 2024 [42]	Spline-based edge activations	Gradient descent with backpropagation; layer freezing	High accuracy, parameter-efficient, adaptable to complex data
U-KAN June 2024 [43]	Encoder-decoder with tokenized KAN layers	Cross-entropy & Dice loss for segmentation; MSE for generation	High interpretability, efficient, adaptable for segmentation and generative tasks, improved accuracy
FourierKAN-GCF June 2024 [44]	Fourier-based GCN with Fourier KAN replacing MLP	BPR loss, grid search, message & node dropout	Efficient, strong interaction representation, robust, adaptable, easier training than spline-KAN
FBKANs June 2024 [45]	Domain decomposition, spline-based local KANs per subdomain	Combined data-driven and physics-informed loss, adaptive grids, parallel training	Scalable, noise-robust, compatible with enhancements
BSRBF-KAN June 2024 [46]	Combines B-splines and Gaussian RBFs	15 epochs (MNIST), 25 (Fashion-MNIST), AdamW optimizer	High accuracy, fast convergence, adaptable activations

Gaussian Radial Basis Functions- FastKAN

Radial basis functions (RBFs) [5, 2] are a class of real-valued functions whose value depends solely on the distance from a center point, also known as the radial distance. These basis functions have long been explored in function approximation, machine learning, and pattern recognition tasks. The basic idea behind RBFs is to model a function by combining several radially symmetric functions, each centered at a different point in the input space. The output of an RBF network is a linear combination of these radial basis functions, weighted by adjustable coefficients. Mathematically, an RBF network with N centers can be represented as:

$$f(x) = \sum_{i=1}^N w_i \phi(\|x - c_i\|)$$

where w_i are the adjustable weights or coefficients and ϕ is the radial basis function, which depends on the distance between the input x and a center c_i . The Gaussian function is one of the most widely used RBFs, defined as

$$\phi(r) = \exp\left(-\frac{r^2}{2h^2}\right)$$

where r is the radial distance, and h is a parameter that controls the width or spread of the function

Simply by applying linear transformations can one well align a series of 3-order B-spline bases to Gaussian radial basis, as is shown in Figure 1. Therefore, one can confidently replace the B-spline bases calculations with Gaussian RBFs.

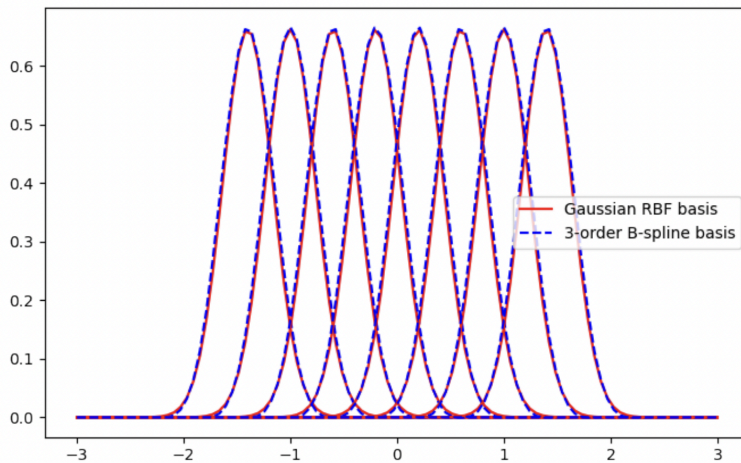


Figure 1: Under proper linear transformations, Gaussian RBFs well approximate 3-order B-spline bases with good precision.

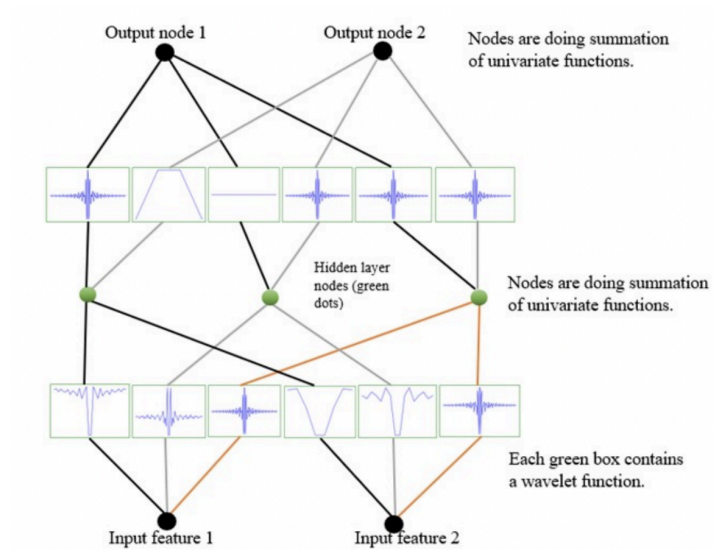


Fig. 1: Wav-KAN with arbitrary number of layers (here is Wav-KAN[2,3,2])

Wav-KAN addresses these limitations by incorporating wavelet functions into the Kolmogorov-Arnold network structure, enabling the network to capture both high-frequency and low-frequency components of the input data efficiently.

Source:

Zavareh Bozorgasl and Hao Chen. Wav-kan: Wavelet kolmogorov-arnold networks. arXiv preprint arXiv:2405.12832,2024

Ziyao Li. Kolmogorov-arnold networks are radial basis function networks. arXiv preprint arXiv:2405.06721, 2024

Activation Function:

What is my decision for activation functions:

```
add_symbolic('gauss', gaussian)
add_symbolic('rbf_c', rbf_c)
add_symbolic('mexhat', mexhat)
```

Gaussian,
Mexican hat,
Shifted Gaussian, PSF-like

First layer edges: assigned physics-inspired bases.

- **Gaussian-like** → resembles the emitter's PSF core.
- **Mexican hat (Laplacian-of-Gaussian)** → models the ring-like residuals around emitters after background subtraction.

Deeper layers: remain cubic B-splines (default of KAN) for flexibility.

Gaussian → captures the core intensity peak.

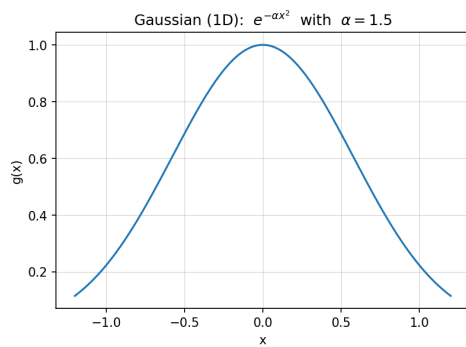
Mexican hat → detects contrast vs. noise.

So my choices is not just B-spline functions:

By seeding the network with **physics-inspired bases** (PSF-like), this makes learning easier and faster.

Splines still exist deeper in the net → they refine details.

1. Gaussian (PSF-core-like)

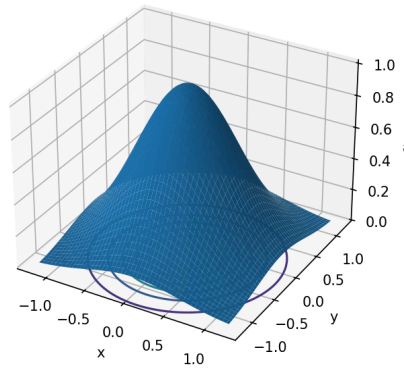
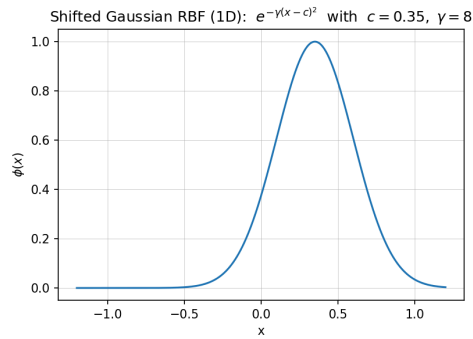


$$g(x; \sigma) = \exp\left(-\frac{x^2}{2\sigma^2}\right)$$

2. Shifted Gaussian RBF (center at ccc)

$$\phi(x; c, \sigma) = \exp\left(-\frac{(x - c)^2}{2\sigma^2}\right)$$

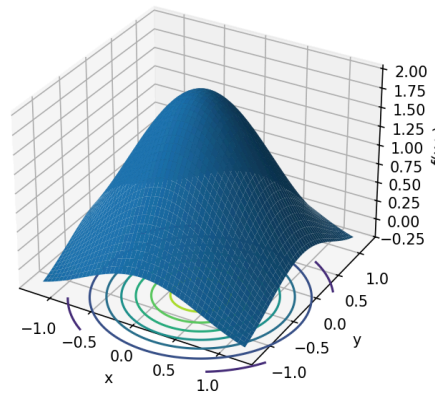
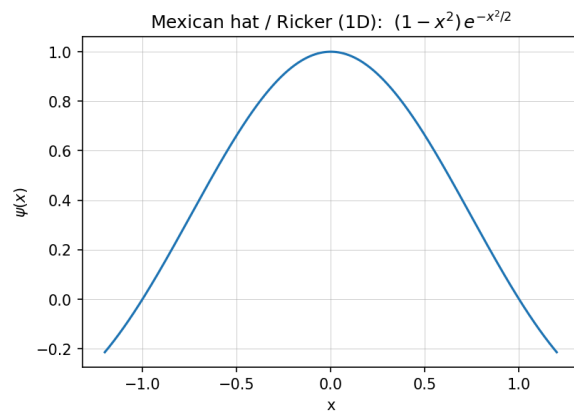
Gaussian (2D surface): $e^{-\alpha(x^2 + y^2)}$, $\alpha = 1.5$



3) Mexican hat / Ricker / (1D) Laplacian-of-Gaussian

$$\psi(x; \sigma) = \left(1 - \frac{x^2}{\sigma^2}\right) \exp\left(-\frac{x^2}{2\sigma^2}\right)$$

Mexican hat / 2D LoG-like: $(2 - (x^2 + y^2)) e^{-(x^2 + y^2)/2}$



Model layers:

First I used 5 layers but the running session crashed so I change it to the standard suggested KAN to 3 layers.

Total layers: 3

1. Input layer → 2) Hidden layer → 3) Output head

Hidden layers: 1

Input features (D): `PATCH_SIZE = 32×32 = 1024`

`width = [1024, 32, 3]`

`Conf_logit` is a number models gives us as its confidence for emitters existence but we need the probability for result so we get sigmoid of this number

- `conf = sigmoid(conf_logit)`

POS_PER_CHUNK = 1200,

NEG_PER_CHUNK = 1200

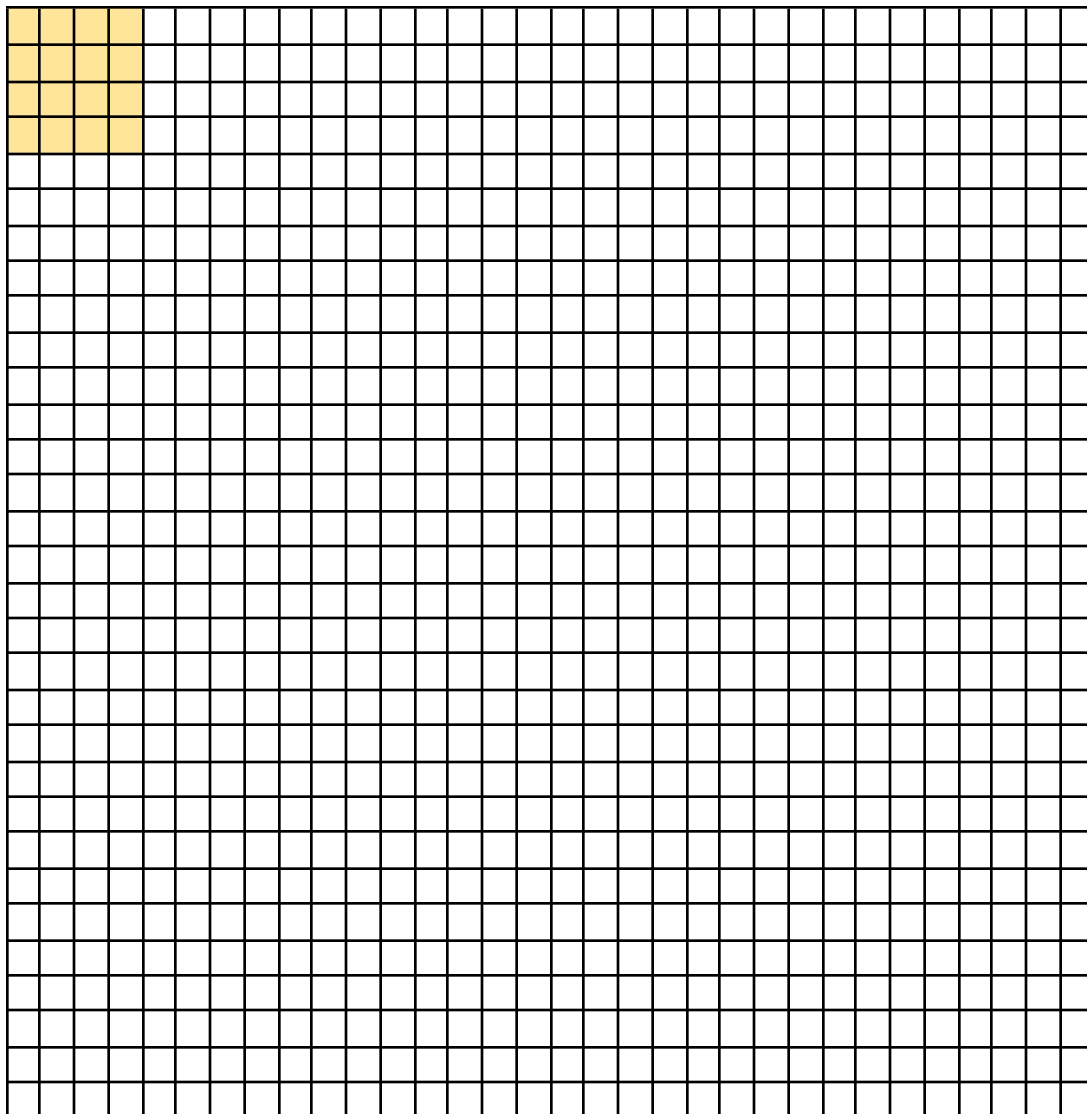
For each CHUNK -> running a function for crops compared to ground truth to get the confidence number if there existed or not?

Suppose each frame has 20 to 40 positive emitters $N_{pos}=24,000$ positive emitter rows to use →
 $num_chunks = (24000 / 1200) = 20$

Chunk is mini batch size because we do cropping

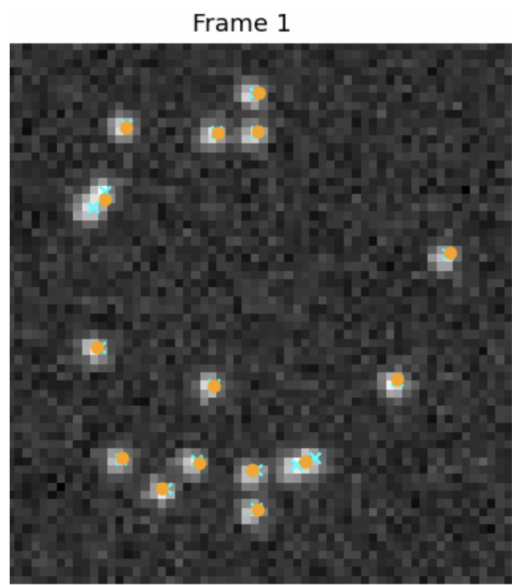
In each frame we have 8*8 patches 4*4 px size and we do positive and negative evaluation

Cropping: our frame is 32 by 32 pixels



Result:
Fram 1

Frame 1: TP=15 FP=0 FN=2 precision=1.000 recall=0.882									
	frame	status	gt_x_px	gt_y_px	est_x_px	est_y_px	conf	err_px	err_min
0 ut actions	1	MATCH	33.217929	53.125807	33.316165	53.157525	0.999937	0.103229	10.322897
	1	MATCH	28.188438	23.205597	28.170612	23.007408	0.999959	0.198989	19.898943
2	1	MATCH	49.918592	23.579395	49.931407	23.784015	0.999923	0.205020	20.502048
3	1	MATCH	17.142016	14.405936	17.344180	14.459253	0.999134	0.209076	20.907641
4	1	MATCH	28.428939	53.022821	28.650194	52.971160	0.999985	0.227206	22.720603
5	1	MATCH	33.259965	8.447023	33.431854	8.276416	0.999984	0.242183	24.218256
6	1	MATCH	14.557805	27.524664	14.281785	27.625711	0.999982	0.293935	29.393457
7	1	MATCH	17.728886	53.944761	17.704601	53.585607	0.999972	0.359974	35.997376
8	1	MATCH	55.967964	38.522637	56.209154	38.818531	0.999966	0.381740	38.174039
9	1	MATCH	22.425191	10.829539	21.967130	10.872514	0.999982	0.460072	46.007215
10	1	MATCH	33.021728	57.707095	33.487553	57.705762	0.999930	0.465827	46.582690
11	1	MATCH	26.106754	14.190555	26.435883	13.841483	0.999913	0.479767	47.976700
12	1	MATCH	33.163385	12.997034	32.667367	13.136895	0.999993	0.515359	51.535854
13	1	MATCH	15.274104	46.048749	15.309993	45.079246	1.000000	0.970167	97.016682
14	1	MATCH	37.898078	13.707509	38.966836	14.043200	1.000000	1.120237	112.023725
15	1	FN	40.161750	14.500692	NaN	NaN	NaN	NaN	NaN
16	1	FN	13.769374	44.180418	NaN	NaN	NaN	NaN	NaN



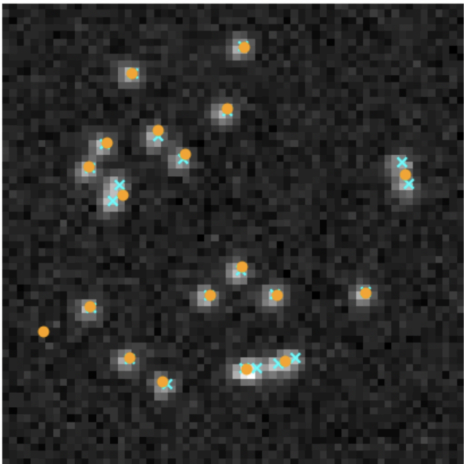
Frame2.

Frame 2: TP=18 | FP=1 | FN=4 | precision=0.947 | recall=0.818

	frame	status	gt_x_px	gt_y_px	est_x_px	est_y_px	conf	err_px	err_nm
0	2	MATCH	17.142016	14.405936	17.144810	14.501682	0.999942	0.095786	9.578598
1	2	MATCH	49.918592	23.579395	50.021849	23.449833	0.999920	0.165676	16.567569
2	2	MATCH	33.021728	57.707095	33.079905	57.536675	0.999974	0.180076	18.007625
3	2	MATCH	28.188438	23.205597	28.373687	23.218416	0.999980	0.185692	18.569176
4	2	MATCH	33.163385	12.997034	33.358722	12.983815	0.999999	0.195784	19.578380
5	2	MATCH	11.885287	21.396101	11.712119	21.516430	0.999921	0.210870	21.087002
6	2	MATCH	17.728886	53.944761	17.575886	53.798671	0.999979	0.211545	21.154539
7	2	MATCH	11.742191	40.738288	11.595481	40.961278	0.996792	0.266925	26.692451
8	2	MATCH	30.814541	48.657733	30.789537	48.925613	0.999992	0.269044	26.904415
9	2	MATCH	37.417479	23.255772	37.691675	23.194112	0.999878	0.281044	28.104384
10	2	MATCH	13.769374	44.180418	14.087507	44.280350	0.999931	0.333459	33.345937
11	2	MATCH	32.605452	26.619482	32.746828	27.140776	0.999886	0.540124	54.012408
12	2	MATCH	24.579094	42.129147	24.848899	42.622983	0.999961	0.562733	56.273332
13	2	MATCH	22.425191	10.829539	21.823313	11.218060	0.999998	0.716384	71.638362
14	2	MATCH	21.149167	45.280578	21.105916	46.048815	0.999990	0.769454	76.945362
15	2	MATCH	37.898078	13.707509	38.745888	14.056226	0.999973	0.916725	91.672505
16	2	MATCH	55.967964	38.522637	55.470221	39.910909	0.999998	1.474804	147.480376
17	2	MATCH	15.712056	38.480248	16.297581	37.057215	0.999997	1.538786	153.878611
18	2	FN	14.882103	36.230086	NaN	NaN	NaN	NaN	NaN
19	2	FN	34.759648	13.101008	NaN	NaN	NaN	NaN	NaN
20	2	FN	40.161750	14.500692	NaN	NaN	NaN	NaN	NaN
21	2	FN	55.015999	41.575592	NaN	NaN	NaN	NaN	NaN
22	2	FP	NaN	NaN	5.273896	18.137971	0.750138	NaN	NaN



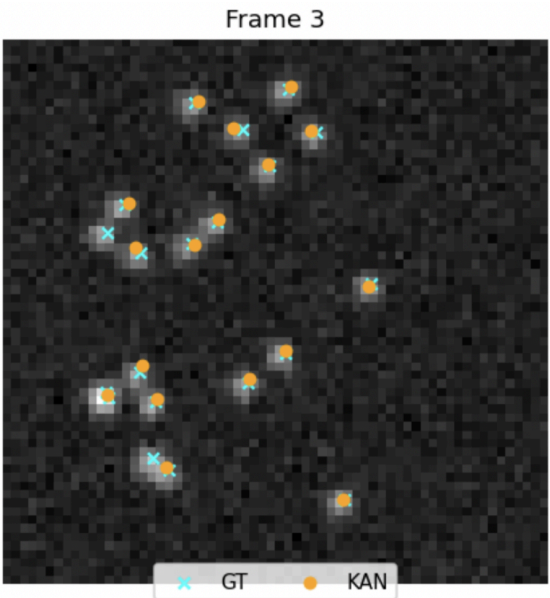
Frame 2



Frame 3.

Frame 3: TP=17 | FP=0 | FN=3 | precision=1.000 | recall=0.850

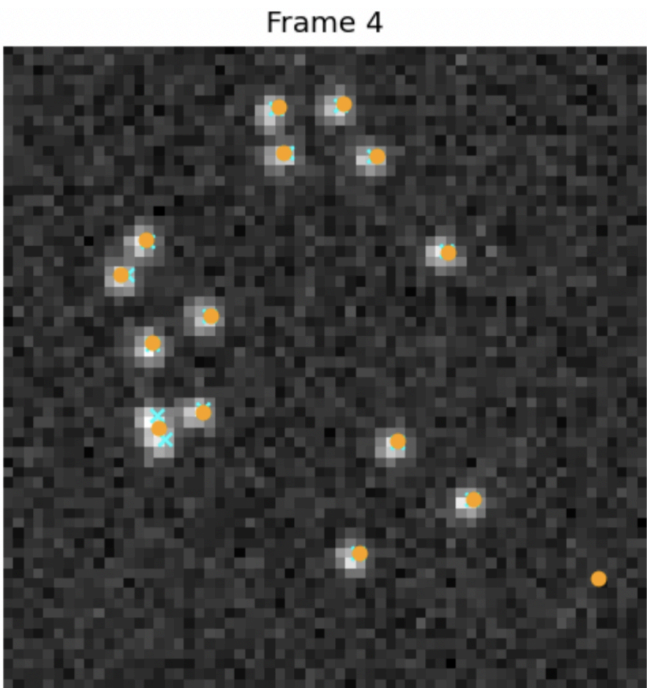
	frame	status	gt_x_px	gt_y_px	est_x_px	est_y_px	conf	err_px	err_nm	
0	3	MATCH	39.561080	9.452121	39.459934	9.490233	0.999960	0.108089	10.808853	
1	3	MATCH	30.814541	48.657733	30.596629	48.786672	0.999940	0.253201	25.320143	
2	3	MATCH	32.605452	26.619482	32.667514	26.870761	0.999977	0.258830	25.882987	
3	3	MATCH	24.579094	42.129147	24.784534	42.365744	0.999188	0.313343	31.334302	
4	3	MATCH	17.396877	20.958827	17.578458	21.267938	0.999887	0.358499	35.849858	
5	3	MATCH	42.823736	34.752414	42.521360	34.488385	0.999999	0.401425	40.142539	
6	3	MATCH	11.548710	22.121853	11.792368	21.796681	0.999990	0.406332	40.633228	
7	3	MATCH	21.614522	39.467513	22.005073	39.318398	0.999961	0.418049	41.804934	
8	3	MATCH	22.031321	56.058084	22.397115	56.261168	0.999943	0.418388	41.838758	
9	3	MATCH	33.021728	57.707095	33.345554	57.993979	0.999896	0.432626	43.262641	
10	3	MATCH	19.004839	12.894230	18.716663	13.250803	0.999998	0.458465	45.846472	
11	3	MATCH	28.188438	23.205597	28.493982	23.632410	0.999983	0.524906	52.490589	
12	3	MATCH	13.769374	44.180418	14.290098	44.266651	0.999913	0.527817	52.781651	
13	3	MATCH	36.370486	52.593786	35.694799	52.752029	0.999827	0.693969	69.396934	
13	3	MATCH	36.370486	52.593786	35.694799	52.752029	0.999827	0.693969	69.396934	
14	3	MATCH	15.495512	24.469245	15.747358	25.216912	0.733408	0.788944	78.894393	
15	3	MATCH	15.712056	38.480248	15.105417	38.999857	0.999804	0.798751	79.875125	
16	3	MATCH	27.638280	52.900930	26.501000	53.065461	0.999906	1.149119	114.911916	
17	3	FN	11.742191	40.738288	NaN	NaN	NaN	NaN	NaN	
18	3	FN	11.885287	21.396101	NaN	NaN	NaN	NaN	NaN	
19	3	FN	17.142016	14.405936	NaN	NaN	NaN	NaN	NaN	



Frame 4.

Frame 4: TP=14 | FP=1 | FN=1 | precision=0.933 | recall=0.933

	frame	status	gt_x_px	gt_y_px	est_x_px	est_y_px	conf	err_px	err_nm
0	4	MATCH	13.769374	44.180418	13.730511	44.205608	0.999142	0.046312	4.631214
1	4	MATCH	14.365234	33.888081	14.297190	33.965746	0.999960	0.103256	10.325618
2	4	MATCH	34.759648	13.101008	34.908886	13.061643	0.999991	0.154342	15.434175
3	4	MATCH	46.128667	18.456777	46.264470	18.349472	0.999810	0.173081	17.308066
4	4	MATCH	38.769049	24.109812	38.736498	24.310122	0.999978	0.202937	20.293743
5	4	MATCH	19.925822	36.758451	20.126783	36.724123	0.999991	0.203872	20.387183
6	4	MATCH	27.638280	52.900930	27.432349	52.852417	0.999998	0.211568	21.156806
7	4	MATCH	43.611353	43.120946	43.769091	42.929768	0.999810	0.247851	24.785118
8	4	MATCH	36.370486	52.593786	36.625432	52.538623	1.000000	0.260846	26.084592
9	4	MATCH	19.284056	27.388042	19.403715	27.108071	0.999895	0.304470	30.446989
10	4	MATCH	26.647940	57.347760	26.937440	57.474752	0.999982	0.316129	31.612852
11	4	MATCH	33.021728	57.707095	33.319234	57.843368	0.999970	0.327231	32.723115
12	4	MATCH	11.742191	40.738288	11.071940	40.767798	0.999692	0.670901	67.090068
13	4	MATCH	15.495512	24.469245	14.966770	25.455436	1.000000	1.118991	111.899116
14	4	FN	14.741163	26.745960	NaN	NaN	NaN	NaN	NaN
15	4	FP	NaN	NaN	58.738121	10.528209	0.359315	NaN	NaN



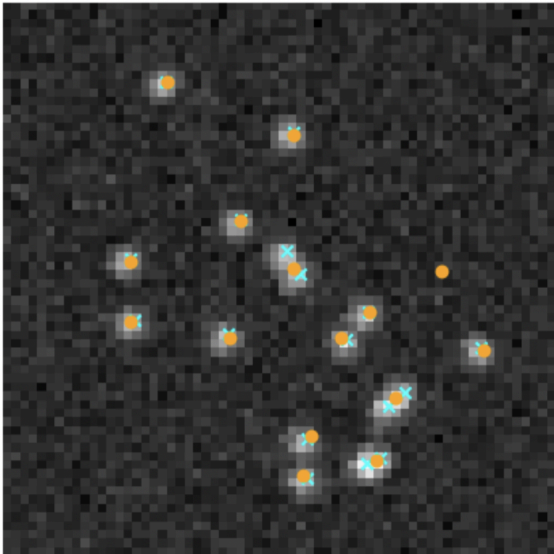
Frame 5.

Frame 5: TP=14 | FP=1 | FN=3 | precision=0.933 | recall=0.824

	frame	status	gt_x_px	gt_y_px	est_x_px	est_y_px	conf	err_px	err_nm
0	5	MATCH	27.160338	38.387961	27.150869	38.258005	0.999995	0.130301	13.030066
1	5	MATCH	41.835029	27.750236	41.991553	27.699345	0.997790	0.164590	16.458981
2	5	MATCH	18.417886	54.351307	18.562475	54.267636	0.999994	0.167054	16.705379
3	5	MATCH	14.365234	33.888081	14.302710	33.577523	0.999988	0.316790	31.678958
4	5	MATCH	25.663849	25.148302	25.772233	24.797601	0.999986	0.367068	36.706781
5	5	MATCH	33.210365	48.551983	33.178483	48.181383	0.999998	0.371969	37.196946
6	5	MATCH	54.922207	23.599016	55.264438	23.303902	0.999993	0.451900	45.190039
7	5	MATCH	14.741163	26.745960	14.277358	26.637070	1.000000	0.476417	47.641654
8	5	MATCH	34.857175	8.537484	34.412546	8.786037	0.999798	0.509385	50.938459
9	5	MATCH	43.275260	10.924784	42.804021	10.565435	1.000000	0.592620	59.261978
10	5	MATCH	39.391207	24.612724	38.792732	24.673084	0.989990	0.601511	60.151136
11	5	MATCH	34.759648	13.101008	35.325565	13.459329	0.999999	0.669818	66.981819
12	5	MATCH	33.961510	32.209787	33.217238	32.746830	0.998765	0.917800	91.779969
13	5	MATCH	44.308217	16.894890	45.109355	17.794114	0.999998	1.204336	120.433622

13	5	MATCH	44.308217	16.894890	45.109355	17.794114	0.999998	1.204336	120.433622
14	5	FN	32.521437	34.810036	NaN	NaN	NaN	NaN	NaN
15	5	FN	46.128667	18.456777	NaN	NaN	NaN	NaN	NaN
16	5	FN	41.819724	10.260938	NaN	NaN	NaN	NaN	NaN
17	5	FP	NaN	NaN	50.430728	32.463179	0.808674	NaN	NaN

Frame 5



System RAM
12.7 GB

Running time: 183 min

Without cropping and mini batch I got the following result:

